

Signal Desk

A working problem-sensing and response pipeline: scan recurring AI-tooling complaints, rank them into an opportunity matrix, then draft (never auto-send) outreach to the people describing the pain.

Team · Divya, Noa, Sreya, Harsh

Course · AI Tools for Venture Workflows, Session 1

How we picked terms and platforms

We did not start with a venture idea. We let the scan surface one, so the picking process itself needed to be disciplined.

Search lexicon

Broad emotion terms alone returned marketing copy, not complaints. We layered three word classes together.

product names Cursor Claude Code Copilot
Operator Comet + symptom words loses context
hallucinate repeats itself + venue filter
site:reddit.com site:news.ycombinator.com

Platforms scanned

- Reddit (search index + direct fetch attempts)
- Hacker News (search index + item pages)
- GitHub Issues on public agent repos
- G2 and review aggregators
- Dev.to and technical Substacks
- X / Twitter · attempted, no free access (see slide 09)

Problem-opportunity matrix

Ranked by a combination of how often the complaint recurs, how much damage it does, and whether we could actually find and message a concentrated group of people describing it.

RANK	PROBLEM CLUSTER	FREQUENCY	SEVERITY	CONCENTRATED IN	OPPORTUNITY READ
01	Coding agents lose context / repeat finished work in long sessions	High	High	GitHub issues, r/ClaudeAI, r/cursor, HN, dev.to	Sharp, technical, and people are already hand-building workarounds. Chosen for the live workflow.
02	"AI agents don't survive production," enterprise reliability skepticism	High	Medium	HN essays, thought-leadership blogs	Loud but diffuse. Hard to turn an essay reader into a discovery call.
03	AI support agents can't escalate or retain ticket context	High	High	G2 reviews, vendor comparison sites	Real pain, but an enterprise sales motion already crowded with incumbents
04	AI browser / computer-use agents too slow for daily use	Medium	Medium	Newsletter and Medium first-person reviews	Pre-chasm category, no clear repeatable wedge yet
05	AI notetakers raise privacy and legal exposure	Medium	Medium	Legal and HR trade press	Policy-adjacent, slower enterprise sales cycle

Context loss in long agentic coding sessions

The people describing this are specific, findable, and already paying for the tools that are frustrating them, which makes them reachable for customer discovery, not just a survey population.

What we actually found

- anthropics/claude-code #92951, "Agent loses context and makes repeated mistakes during extended sessions"
- Dev.to: "Why Your Claude Code Sessions Keep Losing Context (And How to Fix It)"
- Dev.to: "Why Does Claude Code Lose Track Mid-Task?"
- Substack: a user reverse-engineering why /compact discards useful context

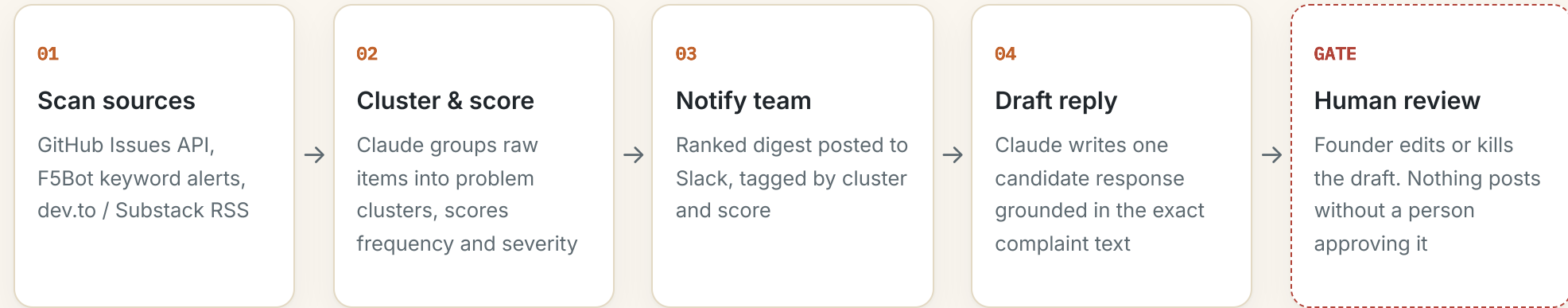
Who is most concentrated here

Developers running multi-day or multi-hundred-tool-call agent sessions on Claude Code, Cursor, Windsurf, and Copilot, most visible in GitHub issue threads on those repos and in r/ClaudeAI and r/cursor.

The tell that this is a real wedge, not just noise: people are already hand-rolling fixes (checkpoint files, manual spec re-statements) instead of waiting for the vendor.

Scan, cluster, alert, draft, then a human decides

Everything up to the send button is automatable. The send button is not.



What we picked at each stage, and why

Updated from what we planned to what is actually deployed and running at signal-desk-dashboard.vercel.app.

STAGE	TOOL	WHY THIS ONE	COST
Scan (technical)	GitHub Issues, Stack Exchange APIs	Direct, official APIs, no scraping, no ToS risk	Free
Scan (social)	HN Algolia, Reddit JSON	Official/public search endpoints; Reddit is best-effort, see slide 09	Free
Scan (community)	Substack RSS, Discourse /search.json	Every Substack exposes an RSS feed; Discourse forums (Cursor's, OpenAI's) expose a free public search API — no scraping needed	Free
Cluster & score, Draft reply	Claude, via OpenRouter	One API key routes to Claude; swappable model slug without touching code	Pay-as-you-go, cents per scan
Notify	Slack + Discord webhooks	Both wired, either optional — Discord needed zero prerequisite setup	Free
Schedule, storage, hosting	Vercel Cron, Vercel Blob, Vercel	Serverless function on a timer, JSON results persisted between runs, auto-deployed from GitHub	Free tier

The two prompts doing the real work

PROMPT A: CLUSTER AND SCORE — LIVE VERSION, RETURNS JSON

You are a research analyst screening startup problem signal.

For each raw item I give you (from GitHub issues, forums, or reviews):

1. Assign it to a problem cluster. Name the cluster as a specific failure in a job-to-be-done, not a vague category.
2. Write a robust 2-3 sentence DESCRIPTION of the problem: what actually breaks, the likely root cause if inferable, and who is affected. Be concrete, cite the mechanism, not the title.
3. Score FREQUENCY 1-5, how often this same complaint recurs.
4. Score SEVERITY 1-5, how much workflow, financial, or trust damage the complaint describes.
5. Note WHERE CONCENTRATED, platform, subreddit, or repo.
6. Flag it if the poster describes a workaround they built themselves. That is a stronger signal than the complaint alone.

Respond with ONLY a JSON array, no prose, matching this shape:

```
[{ "cluster", "description", "frequency", "severity",  
  "concentratedIn", "hasWorkaround", "itemIndexes" }]
```

Raw items (indexed): [live-fetched items from GitHub, HN, Reddit, Stack Overflow, dev.to, Substack, Discourse]

PROMPT B: DRAFT A REPLY (ASTROTURF GUARDRAILS BUILT IN)

Draft a reply from [founder name], building tooling for [cluster].

Here is the exact complaint and source link: [paste complaint + link]

Rules:

- Under 80 words. Sign with the founder's real name.
- Reference the specific detail they described, not a summary.
- Offer something useful first (a workaround, a direct answer) before making any ask.
- If asking for time, frame it as customer discovery, never as a pitch or a beta invite.
- Do not claim experience we do not have.
- If the exchange goes past one reply, disclose we are building something in this space.
- No marketing language.

Real drafts, pulled live from the running dashboard

No longer illustrative composites — this is an actual screenshot of signal-desk-dashboard.vercel.app after a real scan: real complaints from GitHub, HN, and dev.to, with real Claude-drafted replies sitting in "awaiting review." Nothing here was sent.

Alert feed · drafted outreach

Real complaints, real Claude-drafted replies. Nothing here is ever sent automatically — approve/discard is a human decision, logged only in your browser for this demo.

[GitHub Issues](#)Awaiting review

FT-003: dispatch to mac-claude-a — acknowledgement required

I saw your dispatch thread for mac-claude-a building the macOS editor shell against runtime-v1.md.

We're building tooling that watches agent checkpoint commits and surfaces exactly which branch revisions each agent reviewed, plus their adaptation decisions—basically audit trails for the "fetch and review every 3–5 minutes" requirement you specified.

Would 15 minutes walking through your current coordination-cli flow help us understand what's missing?

Div

Approve to sendDiscard

[dev.to](#)Awaiting review

AI Is Already Better at Coding Than Most Software Developers

Fast and easy versus astroturfing

The same automation that makes outreach cheap for a founder is what makes it read as manufactured if we are not careful. Here is where we drew the line.

- 01 Every reply discloses a real name and, past one exchange, that we are building something in this space. No sock accounts, no "just a fellow user" framing.
- 02 The draft always offers something before it asks for anything, a workaround, a direct answer, a resource, never an ask-first message.
- 03 Automation stops at the draft. A person reads, edits, and approves every message before it posts or sends. Nothing goes out unattended.
- 04 Volume is capped intentionally. A founder who replies to forty threads a day in identical cadence reads as a bot even when a human wrote every word.
- 05 Every sent message is logged, so the team can check for repetition or drift toward generic, scripted-sounding language over time.

Real limitations we hit building this

Four from the research phase, two more from actually shipping the pipeline to production.

Direct scraping got blocked

Reddit's own search pages blocked us outright, and still return 403 in the deployed pipeline today. Hacker News item pages rate-limited a second request. Confirms you need an official API, not ad hoc fetching.

Broad queries returned marketing, not complaints

"AI agent frustrating" mostly surfaced explainer blogs and thought-leadership. Real complaint density only showed up once we narrowed to a specific product name plus a specific symptom word plus a specific venue.

X was a dead end on the free tier

No free path to X search turned up real posts. The sample tool list treats X API/MCP as a given, but it is a real gap unless someone already has paid access.

Aggregator sites validate a pattern, not a lead

G2 confirmed the support-agent cluster is real at category scale, but gave us nothing individually addressable to message. Good for sizing a problem, useless for sourcing targets.

The model we picked got silently deprecated

The exact OpenRouter model slug we launched with (`claude-3.5-sonnet`) stopped resolving mid-build with a bare 404. Model catalogs move under you; a live pipeline needs a fallback, not a hardcoded slug.

Seven sources, one time limit

Fetching all sources sequentially blew past Vercel's 60s function limit and hit a 504. Fix was parallelizing every per-query request with `Promise.allSettled` — obvious in hindsight, invisible until real load hit it.

This is not a simulation anymore

The demo now runs on the actual deployed product: signal-desk-dashboard.vercel.app. Type any problem area, watch it generate real search queries, fetch live items, and cluster them — in front of the class, live.

Explore a problem area

Type any problem area and Claude will generate real search queries, fetch live items for them, and cluster the results into sub-problem opportunities — on demand, separate from the scheduled scan above.

Find sub-problems

Searched 8 generated queries (Otter.ai transcription inaccurate · Fireflies.ai missing meeting recording · Grain app sync failed · Fathom video recorder crashes · Otter.ai speaker identification wrong · Fireflies.ai bot kicked from meeting · Tactiq transcript incomplete · Otter.ai export broken), found 15 items, clustered into 3 sub-problem areas.

CLUSTER	FREQUENCY	SEVERITY	CONCENTRATED IN	WORKAROUND SEEN?
Otter.ai bot joins meetings without user consent despite opt-out settings Users who explicitly disable Otter.ai's auto-join feature report the bot still joins	3/5	5/5	Hacker News	—

Typed "AI meeting notetaker apps" and it independently surfaced a real Hacker News thread about Otter.ai recording meetings without consent — the same privacy risk flagged in slide 02's research, found again live.

What we would build next

Wiring GitHub, Slack, and a real feed is done — it is live. Here is what is genuinely next.

- Get real Reddit access (official API or a paid tripwire like Syften) instead of the public JSON endpoint that gets blocked in production
- Persist approve/discard decisions server-side instead of only in the browser, so the review log survives a refresh
- Run 10 real drafted replies past a non-founder teammate as an astroturfing smell test before sending any
- Decide, after real discovery conversations, whether this cluster is a venture or just a good feature for someone else's product

SOURCES

[GitHub · anthropics/claude-code issue #92951](#)

[Twig · Most Common Complaints About AI Customer Support Tools](#)

[G2 · AI Agents for Customer Support, 7,900+ reviews](#)

[Nate's Newsletter · I threw my real workday at two AI agents](#)

[Hacker News · Most "AI agents" don't survive production](#)